

TP2 : Putting everything together

avec Express.js

Réalisé par :
TAFTAF Wijdane

Encadré par : Pr. Amal Ourdou

Année Universitaire : 2025 / 2026

Table des matières

Introduction	2
Structure du Code	3
0.1 Structure des dossiers	3
0.2 Technologies Utilisées	3
0.3 Implémentation	3
0.3.1 Connexion à MongoDB	3
0.3.2 Modèle Utilisateur (Mongoose)	3
0.3.3 Configuration Passport.js	4
0.3.4 Protection des routes	4
0.4 Résultat Final	4
Conclusion	6

Introduction

Ce travail pratique a pour objectif de consolider les notions vues en cours sur le développement web backend avec **Node.js** et **Express**. Le but principal du projet est de réaliser une application permettant :

- l'inscription et la connexion des utilisateurs,
- la gestion des sessions ,
- le stockage des utilisateurs dans **MongoDB**,
- la redirection vers une page de livres après authentification,
- l'utilisation du moteur de template **Pug**,
- la sécurisation de l'accès à la page des livres pour les utilisateurs non connectés,
- l'utilisation de **TailwindCSS** pour la mise en forme.

Le thème choisi est : *Application de gestion de livres personnels*. Après authentification, l'utilisateur accède à une page présentant une liste de livres.

Structure du Code

0.1 Structure des dossiers

```
TP2/  
  models/  
    User.js  
  routes/  
    auth.js  
    books.js  
  views/  
    login.pug  
    register.pug  
    books.pug  
    layout.pug  
  public/  
  app.js  
  .env  
  package.json
```

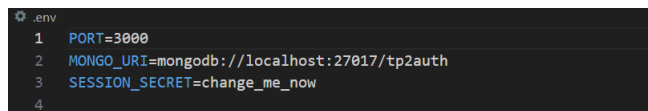
0.2 Technologies Utilisées

- **Node.js** / **Express** : serveur backend
- **MongoDB** + **Mongoose** : base de données NoSQL
- **Pug** : moteur de templates
- **TailwindCSS** : stylisation moderne du front-end
- **bcrypt** : hachage des mots de passe
- **express-session** + **connect-mongo** : sessions utilisateurs

0.3 Implémentation

0.3.1 Connexion à MongoDB

L'application utilise un fichier `.env` pour stocker les variables sensibles.



```
.env  
1 PORT=3000  
2 MONGO_URI=mongodb://localhost:27017/tp2auth  
3 SESSION_SECRET=change_me_now  
4
```

0.3.2 Modèle Utilisateur (Mongoose)

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, required: true }
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

0.3.3 Configuration Passport.js

```
passport.use(new LocalStrategy({ usernameField: 'email' }, async (email, pwd, done) => {
  try {
    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user) return done(null, false, { message: 'Utilisateur introuvable' });
    const ok = await bcrypt.compare(pwd, user.password);
    if (!ok) return done(null, false, { message: 'Mot de passe invalide' });
    return done(null, user);
  } catch (e) { return done(e); }
}));

passport.serializeUser((user, done) => done(null, user.id));
passport.deserializeUser(async (id, done) => {
  try { done(null, await User.findById(id).select('-password')); }
  catch (e) { done(e); }
});
```

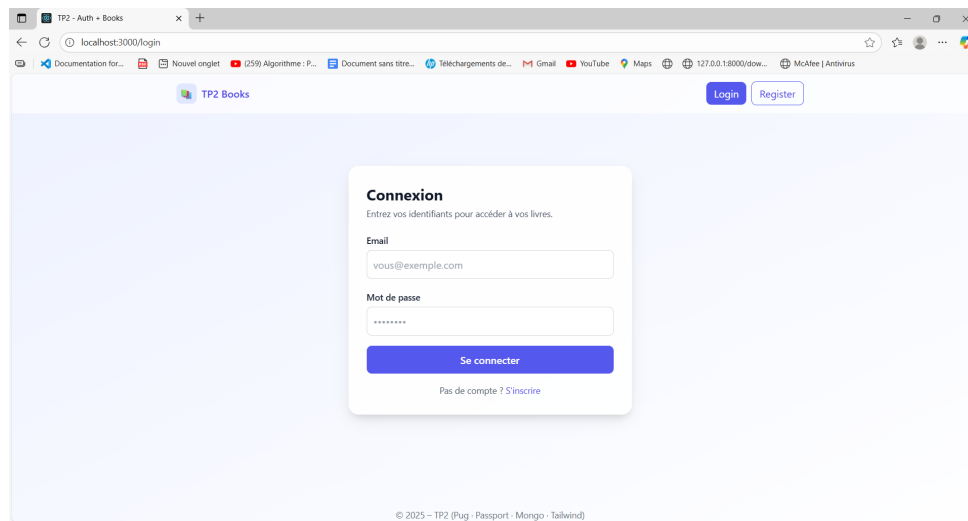
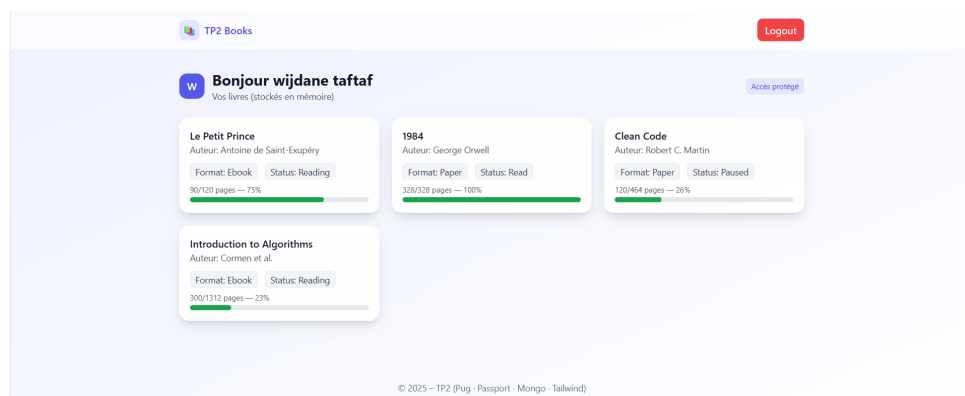
0.3.4 Protection des routes

```
function ensureAuthenticated(req, res, next) {
  if (req.isAuthenticated() && req.isAuthenticated()) return next();
  return res.redirect('/login');
}

router.get('/books', ensureAuthenticated, (req, res) => {
  res.render('books', { books });
});
```

0.4 Résultat Final

Après une connexion réussie, l'utilisateur est redirigé vers la page des livres. Les livres sont stockés dans une variable locale et affichés avec un design épuré grâce à TailwindCSS.

FIGURE 1 – Page *Login* (Pug + Tailwind)FIGURE 2 – Page *Books* protégée après authentificationFIGURE 3 – Document *users* dans MongoDB (mot de passe haché via *bcrypt*).FIGURE 4 – Document de session stocké dans MongoDB (*connect-mongo*).

Conclusion

Ce projet m'a permis de mettre en pratique des concepts essentiels du développement web moderne. J'ai pu comprendre et implémenter une authentification basée sur les sessions en utilisant Passport.js, ainsi que manipuler et gérer des données avec MongoDB à l'aide de Mongoose. Ce travail m'a également permis d'apprendre à sécuriser des routes, gérer des sessions utilisateurs et organiser une application Express de manière professionnelle et modulaire. Par ailleurs, l'intégration de TailwindCSS m'a offert l'opportunité d'appliquer un design moderne, responsive et élégant. Ainsi, ce TP m'a donné une base solide pour concevoir des applications web complètes, sécurisées et adaptées aux standards actuels du développement.